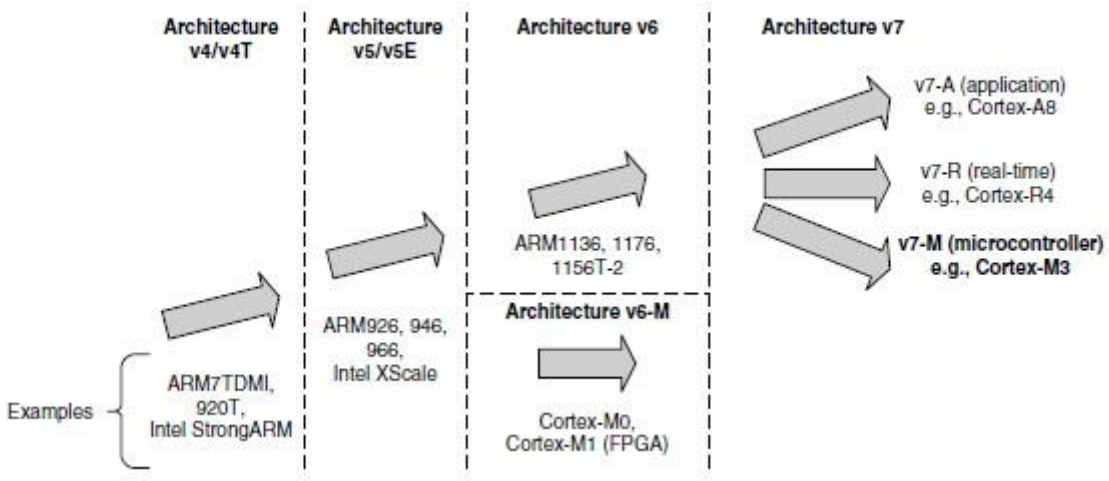
Title: Exploring the ARM Cortex M3
Author: Craig Duffy 6/6/14, 11/01/18, 23/02/2, 26/01/22
Module: Mobile and Embedded Devices, Secure Embedded Systems
Awards: BSc CSI, BSc Forensic Computing, Computer Security.
Prerequisites: Basic computer architecture and some C and assembler knowledge

This worksheet gives an introduction to the ARM Cortex M3 processor and the tools required to access and program it.

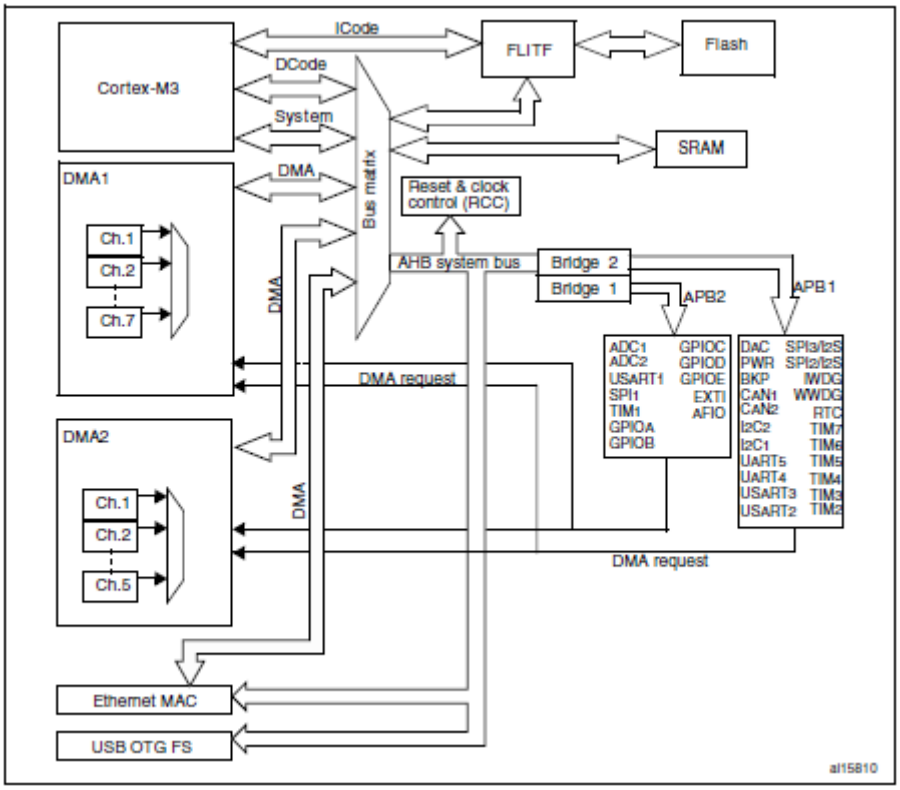
Introducing the ARM Cortex M3

The ARM Cortex M3 is one of a new series of ARM processors that they hope will further cement their leading position as chip designers. The Cortex series has 3 distinct variants – the A, R and M profiles. The A is aimed at the application market and these processors, such as the A9, are found in devices such as tablet computers and smart phones. The R is for real-time processing where latency and speed are key factors, they are used in devices such as hard disk controllers and automotive products. Finally, the M is aimed at the micro controller market - small, efficient, low powered systems with lots of peripherals. If the 'Internet of things' is to become a reality then it will be based on chips like the ARM Cortex M3.



ARM processors and architectures

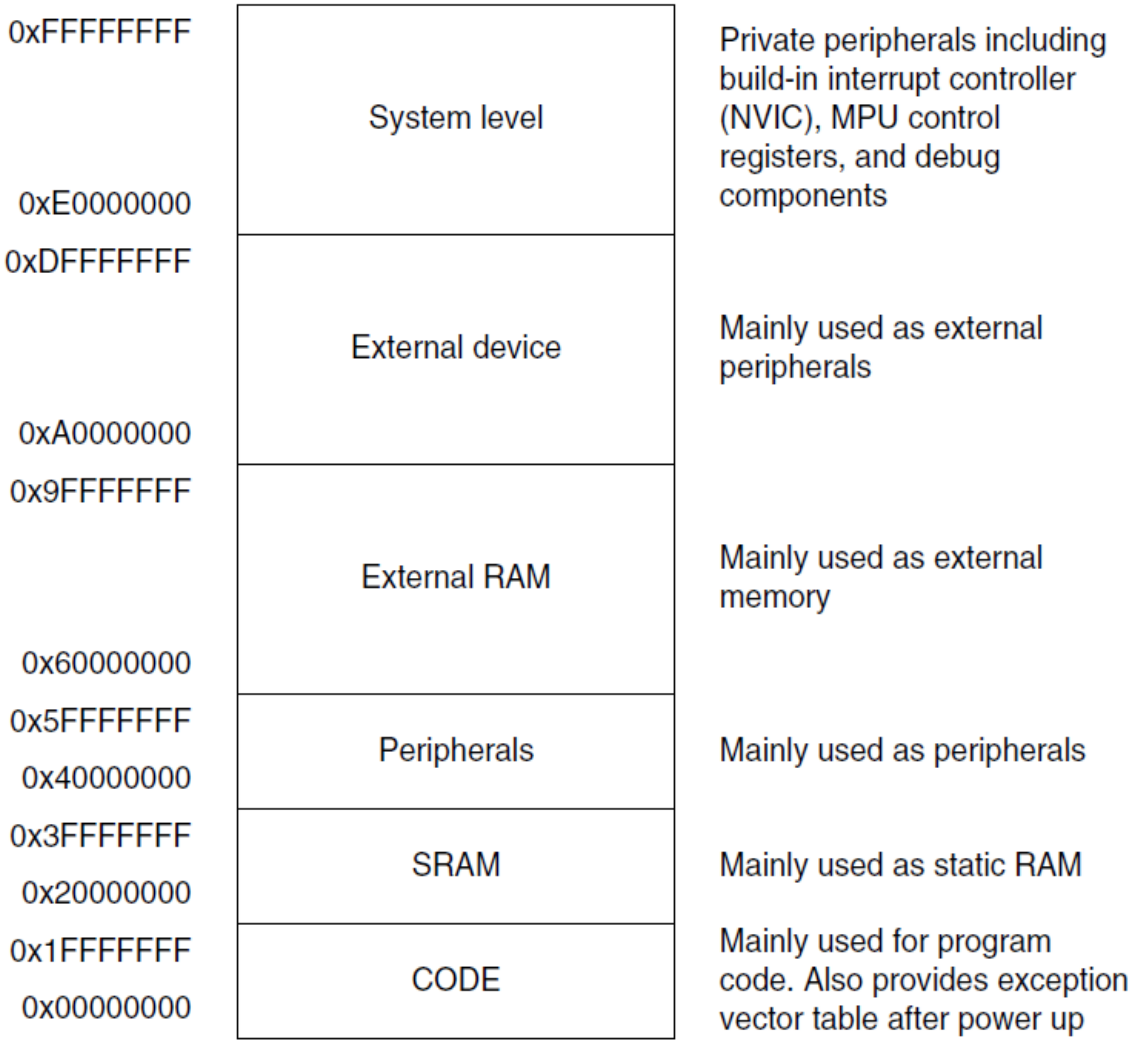
ARM have a rather confusing way of naming their processors and architectures. From the above diagram you can see that the ARM7 – a very popular processor – is architecture version 4 and the Cortex M3 is architecture version 7! The common features of all the ARM processors are: low clock speed, the use of a reduced instruction set (RISC), and very good power/size efficiency. So whilst the M3 is a full 32 bit CPU it uses the Thumb2 instruction set which allows a mixing of 16 and 32 bit instructions. The use of 16 bit instructions increases code density, effectively doubled over 32 bit instructions, but 32 bit instructions can be used when they are needed.



STM3 connectivity line bus architecture

ARM don't make CPUs and systems - they design them and other companies fabricate them. For example ST Micro make a series of ARM Cortex processors, or Texas Instruments make many CPUs based on ARM designs.

In the above diagram we can see the basic architecture of a ST Micro CM3 connectivity line device where ST have added a large number of networking support to ARM's basic design. ST have created many other CM3 variants that stress low clock speed, larger memory foot print and so on. The M series are 'Harvard architecture' machines, meaning that code and data accesses are done on separate buses allowing parallel processing of data and instructions. In the above diagram we can see the separate Instruction and Data buses – ICode and DCode – these are restricted to the Code section – the first 0.5GB of memory. This doesn't mean that code can't be stored and executed in other memory regions, but that this is the most efficient place to execute it from, as the system can interleave data fetches and code execution to maximise efficiency.



ARM M-series memory map

Unlike early ARM processors the CM3 has a fixed 4GB memory map with specific features in specified areas. This fixed memory map is intended to facilitate the easy porting of software from one device to another. The peripheral areas allow the actual device fabricators to put their own peripherals in that area – for example the STM32F107VCT6 is part of a set of M3 devices that ST calls connectivity line devices which strongly feature networking peripherals in their design. We will look in detail at some regions such as the Nested Vectored Interrupt Controller (NVIC), Memory Protection Unit (MPU) and Bit-band regions in detail in later lectures and worksheets. As we shall see ARM provide a full set of libraries and start up code for these devices which are supplemented by software from the device fabricators.

When the system starts up it needs to know what to execute and where its system stack is located in RAM. This information is in the vector table. The first entry in the vector table is the address of some RAM for the initial stack pointer. After that the table holds a series of pointers to code that is executed under various circumstances – for example what code needs executing after a bus error. The first vector after the stack is the Reset Vector this is the first piece of code that is executed at system start up. Other vectors are for important system events like interrupts or timer ticks. After these entries, i.e. vectors from IRQ 16 onwards, the vectors are available for user code. The vector table can be relocated at system start up time thus allowing a mixture of pre-fixed, flash based vectors and dynamic RAM based ones.

Exception Type	Address Offset	Exception Vector
18–255	0x48–0x3FF	IRQ #2–239
17	0x44	IRQ #1
16	0x40	IRQ #0
15	0x3C	SYSTICK
14	0x38	PendSV
13	0x34	Reserved
12	0x30	Debug monitor
11	0x2C	SVC
7–10	0x1C–0x28	Reserved
6	0x18	Usage fault
5	0x14	Bus fault
4	0x10	MemManage fault
3	0x0C	Hard fault
2	0x08	NMI
1	0x04	Reset
0	0x00	Starting value of the MSP

CM3 Vector Table

Questions.

Find out which devices, including your own, have ARM chips in them. Examples can be tablets, mobile phones, music players, televisions, home routers. Who makes the chips?

Cortex M3 Software

We are now going to look at building some code for our boards. Starting up a CPU and a board is complicated – even if you aren't doing anything interesting, which we won't be for a while, a lot of things need setting up and organising. Historically this sort of activity has been a somewhat bespoke folk art, replete with weird traditions and handed down wisdom. Software and hardware developers have been trying over the decades to clean up this process and make it more standardised. However, in order to create such code you still need to know a fair amount about your hardware and about how systems start up.

Part of this standardisation is seen in ARM's creation of a set of code libraries called CMSIS – they pronounce it *SIMSYS* for reasons best known to themselves, CMSIS is Cortex Micro-controller Software Interface Standard. This is in many ways the systems' BIOS. This also includes a set of C source and header files, some assembler code and configuration information. This is normally supplemented by the fabricator's software libraries – for ST Micro this is STM32F10x_StdPeriph_Lib_V3.5.0 which is their library for their CM3 series of CPUs and boards. STM32F10x_StdPeriph_Lib_V3.5.0 contains the CMSIS code (provided by ARM) to control the CPU as well as other library code for all the other peripherals that are supported – there are 26 peripherals in all across the STM series range.

```
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/
├─CMSIS
│  ├─CM3
│  │  ├─CoreSupport
│  │  │  ├─core_cm3.c
│  │  │  └─core_cm3.h
│  │  └─DeviceSupport
│  │     └─ST
│  │        └─STM32F10x
│  │           ├─…
│  │           ├─stm32f10x.h
│  │           ├─system_stm32f10x.c
│  │           └─system_stm32f10x.h
│  └─…
└─STM32F10x_StdPeriph_Driver
   ├─inc
   │  ├─misc.h
   │  ├─…
   │  ├─stm32f10x_gpio.h
   │  ├─stm32f10x_i2c.h
   │  ├─stm32f10x_spi.h
   │  ├─stm32f10x_usart.h
   │  └─…
   └─src
      ├─misc.c
      ├─…
      ├─stm32f10x_gpio.c
      ├─stm32f10x_i2c.c
      ├─stm32f10x_spi.c
      ├─stm32f10x_usart.c
      └─…
```

ST Standard Peripheral Library including CMSIS

To complete this worksheet you are going to need to clone the following git repo https://gitlab.uwe.ac.uk/c-duffy/ses-worksheet-2.git. If you aren't able to use git please see the note about untarring in the appendix.

This directory has 2 main subdirectories: the system code and our worksheet code. For this worksheet, like the previous you won't need to submit any work, so you only need to clone. The system code is the code outlined above, and is provided by STMicro and ARM. The main part of our work for most of this worksheet is going to be in the worksheet2 directory. This contains the following files:

main.c Makefile Makefile~ openocd.cfg startup_stm32f10x.c stm32f100.ld stm32f10x_conf.h

The openocd.cfg file has been covered in worksheet1. The best place to start understanding what is going on is through the Makefile – a full listing is given in Appendix A. This is a command file for the **make** command. This command is used to build and configure systems. The **makefile** consists of a series of comments, variables, labels, commands and rules. The comments are started by the # character and they are there to aid or confuse the human reader. Some of the variables such as:

TOOLROOT=/usr/bin

are telling the system where to find things, some, such as,

```
CC=$(TOOLROOT)/arm-none-eabi-gcc
```

are redefining internal variables thus telling the makefile which C compiler is being used – the $ is used to access an existing variable.

Built in **make** commands such as,

```
vpath %.c .
```

are telling **make**, and by implication all of the tool chain, where to search for important files. This is necessary when doing cross development, normal development can rely on looking in standard places for various files used to build executables, something we can't rely upon.

There are then a series of flags which are to be used by the cross compiler to make our executable - += means add to an existing variable :

```
CFLAGS+= -mcpu=cortex-m3 -mthumb
```

These flags are specifying the type of output the compiler will generate. Again this is of great importance to cross developers as we can rarely rely on the default settings.

Question.

Use the **man gcc** command to find out what these command flags mean. Find out how to stop the compilation process at various stages, explaining what they are. How do you build a version of the binary for a debugger? Find out how many ARM CPUs are supported.

There is also a series of labels. These are used by **make** to figure out what to do, in what order and under what rules. So:

```
$(ELF) : $(OBJS)
$(LD) $(LDFLAGS) -o $@ $(OBJS) $(LDLIBS)
```

This will translate into:

```
demo.elf: startup_stm32f10x.o system_stm32f10x.o main.o
gcc -T stm32f100.ld  -mthumb -mcpu=cortex-m3
```

Which means: in order to make our target, demo.elf, we require the object files startup_stm32f10x.o system_stm32f10x.o and main.o, once we have those we execute the C compiler (gcc). This is known as the default rule. What happens if these files, **main.o** etc., don't exist? Well **make** can create them. It has a general set of rules on how to make .o files from C source files (**.c**) and assembler source files (**.s**).

```
%.o: %.c

$(CC) -c $(CFLAGS) $< -o $@
$(CC) -MM $(CFLAGS) $< > $*.d

%.o: %.s
$(CC) -c $(CFLAGS) $< -o $@
```

There is one more complication, building object or executable files requires a certain order for file inclusion and has strict dependencies. The line **$(CC) -MM $(CFLAGS) $< > $*.d** deals with this. It creates a series of .d files containing the dependencies for compilation and these files are included as rules here:

-include $(OBJS:.o=.d)

This allows **make** to look through the various path names it accesses to find and create the files and dependencies it requires in order to create **demo.elf**, the binary that we wish to download onto our board. Once you have built **demo.elf** you can look at the contents of **main.d** and so on.

There are a couple of other labels – **clean** and **debug**, that can be called from the command line. I will leave it for you as an exercise to figure what these labels are for and what they do.

The principle C file, **main.c**, has very little in it as it is just a demo for us to download on to the board and to execute it under the control of the GNU debugger, **gdb**.

```
int i = 0;
int off = 5;
void inc(void){
  i += off;
}
int main(void){
  while (1) {
    inc();
  }
}
```

main.c

The two other files st**a**rtup_stm32f10x.c and system_stm32f10x.c are very important system files. The file startup_stm32f10x.c has the reset vector for the system, namely the first piece of code that is executed on system start up. This function initialises all of the memory regions that any running program is going to need in order to execute. This file also has the definitions of all of the system vectors for the device. It calls a function, SystemInit, which is in the other file, system_stm32f10x.c, this is supplied as part of the ST Micro standard peripheral library. It is in STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/system_stm32f10x.c. This file does all of the initialisation of the board and device, it knows exactly which device we have through the makefile variable PTYPE = STM32F10X_CL, showing that we are using the connectivity line device. Once this code, specifically the code in the function SystemInit, has been executed the device's various clocks and timers, and the memory map are initialised.

You will notice that these files are all C code which is unusual in these circumstances, normally one would expect this level of code to be in assembler. ARM are deliberately trying to keep as much work in a high level language like C rather than an assembler. There are in fact some assembler routines required for start up in the CMSIS library, the one for our board is called startup_stm32f10x_cl.s, but this code is fairly well hidden and changes to it are unlikely.

One area where a detailed knowledge is essential is creating a bootable executable. This means laying the code out in the correct format for the board, ensuring that certain routines are in specific places and will be executed in a certain order. The linker and the initialisation code work together to ensure that the code has an adequate run time environment. On non cross developed systems this area is invisible to all except system developers and, of course, hackers. In the Makefile above we saw that to create the executable **demo.elf** required the command line:

```
gcc -T stm32f100.ld  -mthumb -mcpu=cortex-m3 startup_stm32f10x.o system_stm32f10x.o main.o
```

The **-T** option to the compiler specifies the name of the script that the linker will use to build the executable – the script is given in full in Appendix B. This file, **stm32f100.ld**, is a linker description script which tells the linker what resources are available on the board and how to organise them.

Early on in the script it has the directive:

```
ENTRY(Reset_Handler)
```

This tells the linker where the entry point for the code is which means the code that will be executed first on system start up. Other important information is about the memory resources in the system. The Olimex board has an STM32F107VCT6 CM3 chip which has 256KB of Flash and 64KB of SRAM. The Flash memory, effectively ROM memory although it can be re-written using special commands through OpenOCD, is at address 0x08000000, the SRAM, which is volatile and can be written to, is at address 0x20000000. The directive:

```
MEMORY
{
 RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 64K
 FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 256K
}
```

informs the linker of the memory regions' sizes.

Various labels are set up to mark places in the memory map.

```
__ram_start__ = ORIGIN(RAM);
__ram_size__  = LENGTH(RAM);
__ram_end__   = __ram_start__ + __ram_size__;
_estack = __ram_end__;
```

These labels are available in the C code. The script then goes on to specify where each part of the executable is placed in memory.

```
.data : AT ( _sidata )
{
...
} >RAM
```

The labels are **.text** for executable code, **.data** for initialised data and **.bss** for uninitialised data. You will notice that some data is placed in flash to be copied into RAM on start up. This code is in the C file startup_stm32f10x.c:

```
void Reset_Handler(void) {
   unsigned long *src, *dst;

   src = &_sidata;
   dst = &_sdata;

   // Copy data initializers
   while (dst < &_edata)
    *(dst++) = *(src++);
```

A large amount of the script is taken up with labels from the symbol table (**.stab**) that are for debugging.

To build the application is very simple. In the worksheet2 directory type in the **make** command.

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm32...  craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm32...  craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm32/...  craig@craig-HP-Compaq-8200-Elite-SFF-PC: ~/stm32/...
source [find target/stm32f1x.cfg]
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ file demo.elf
demo.elf: ELF 32-bit LSB executable, ARM, version 1 (SYSV), statically linked, not stripped
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ ls -l demo.elf
-rwxrwxr-x 1 craig craig 76521 Jun 23 12:03 demo.elf
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ more main.c
int i = 0;
int off = 5;

void inc(void){
  i += off;
}


int main(void){
  while (1) {
    inc();
  }
}

craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ make
make: `demo.elf' is up to date.
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ make clean
rm -f startup_stm32f10x.o system_stm32f10x.o  main.o startup_stm32f10x.d system_stm32f10x.d main.d demo.elf
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ make
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -c -O1 -g -mcpu=cortex-m3 -mthumb  -I. -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/Device
0x -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/STM32F10x_StdPeriph_Driver/inc -I. -DSTM32F1
IPH_DRIVER -DUSE_FULL_ASSERT  startup_stm32f10x.c -o startup_stm32f10x.o
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -MM -O1 -g -mcpu=cortex-m3 -mthumb  -I. -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/Devic
10x -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/STM32F10x_StdPeriph_Driver/inc -I. -DSTM32F
RIPH_DRIVER -DUSE_FULL_ASSERT  startup_stm32f10x.c > startup_stm32f10x.d
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -c -O1 -g -mcpu=cortex-m3 -mthumb  -I. -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/Device
0x -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/STM32F10x_StdPeriph_Driver/inc -I. -DSTM32F1
IPH_DRIVER -DUSE_FULL_ASSERT  ../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/system_stm32f10x.c -o system_stm32f10x.o
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -MM -O1 -g -mcpu=cortex-m3 -mthumb  -I. -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/Devic
10x -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/STM32F10x_StdPeriph_Driver/inc -I. -DSTM32F
RIPH_DRIVER -DUSE_FULL_ASSERT  ../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/system_stm32f10x.c > system_stm32f10x.d
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -c -O1 -g -mcpu=cortex-m3 -mthumb  -I. -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/Device
0x -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/STM32F10x_StdPeriph_Driver/inc -I. -DSTM32F1
IPH_DRIVER -DUSE_FULL_ASSERT  main.c -o main.o
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -MM -O1 -g -mcpu=cortex-m3 -mthumb  -I. -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/Devic
10x -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport -I../STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/STM32F10x_StdPeriph_Driver/inc -I. -DSTM32F
RIPH_DRIVER -DUSE_FULL_ASSERT  main.c > main.d
/root/CodeSourcery/Sourcery_G++_Lite/bin//arm-none-eabi-gcc -Tstm32f100.ld -mthumb -mcpu=cortex-m3  -o demo.elf startup_stm32f10x.o system_stm32f10x.o  main.o
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$
```

make output

<u>Downloading and running the program</u>

In order to run and debug the program you are going to need a number of command terminals open with the worksheet2 directory as its current working directory. This directory is the one with the demo program, makefiles and code in it. You need to be in this directory for the various tools to find the code they are going to be working on, you can check if you are in it either by listing the contents – **ls** – or by using **pwd (**print working directory). Now make sure that the board is connected via the USB JTAG dongle and is powered up, namely the red power led is on.

Open the first terminal and run the **openocd** command you should see some output like:

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ !open
openocd -f openocd.cfg
Open On-Chip Debugger 0.7.0 (2014-02-11-17:13)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.sourceforge.net/doc/doxygen/bugs.html
Info : only one transport option; autoselect 'jtag'
adapter speed: 1000 kHz
adapter_nsrst_delay: 100
jtag_ntrst_delay: 100
cortex_m3 reset_config sysresetreq
Error: unable to open ftdi device: device not found
in procedure 'init'
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ openocd -f openocd.cfg
Open On-Chip Debugger 0.7.0 (2014-02-11-17:13)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.sourceforge.net/doc/doxygen/bugs.html
Info : only one transport option; autoselect 'jtag'
adapter speed: 1000 kHz
adapter_nsrst_delay: 100
jtag_ntrst_delay: 100
cortex_m3 reset_config sysresetreq
Info : max TCK change to: 30000 kHz
Info : clock speed 1000 kHz
Info : JTAG tap: stm32f1x.cpu tap/device found: 0x3ba00477 (mfg: 0x23b, part: 0xba00, ver: 0x3)
Info : JTAG tap: stm32f1x.bs tap/device found: 0x06418041 (mfg: 0x020, part: 0x6418, ver: 0x0)
Info : stm32f1x.cpu: hardware has 6 breakpoints, 4 watchpoints
Info : accepting 'telnet' connection from 4444
```

openocd output

Open another command window in the directory and run the command **telnet localhost 4444**. This gives us a **telnet** connection to the board. In order to debug the code we need it to be loaded onto the board, and as the code is targeted at flash on the board we require special commands to load it. For the moment we are going to do this with **telnet** in a later worksheet I will show how we can create a PERL script to do this from within **gdb**. Once **telnet** is running and connected to the board type in the following commands – **in bold –** at the **telnet** command prompt >. The output in *italics* is from the board.

> reset halt

JTAG tap: stm32f1x.cpu tap/device found: 0x3ba00477 (mfg: 0x23b, part: 0xba00, ver: 0x3)
JTAG tap: stm32f1x.bs tap/device found: 0x06418041 (mfg: 0x020, part: 0x6418, ver: 0x0)
target state: halted
target halted due to debug-request, current mode: Thread xPSR: 0x01000000 pc: 0x08000190 msp: 0x20002000

> stm32f1x mass_erase 0

stm32x mass erase complete

> flash write_bank 0 demo.elf 0

odd number of bytes to write, padding with 0xff
wrote 76521 bytes from file demo.elf to flash bank 0 at offset 0x00000000 in 2.627453s (28.441 KiB/s)

Running telnet and loading **demo.elf**.

This now means that **demo.elf** is loaded onto the board. If you don't understand any of this then you need to rework through worksheet 1!

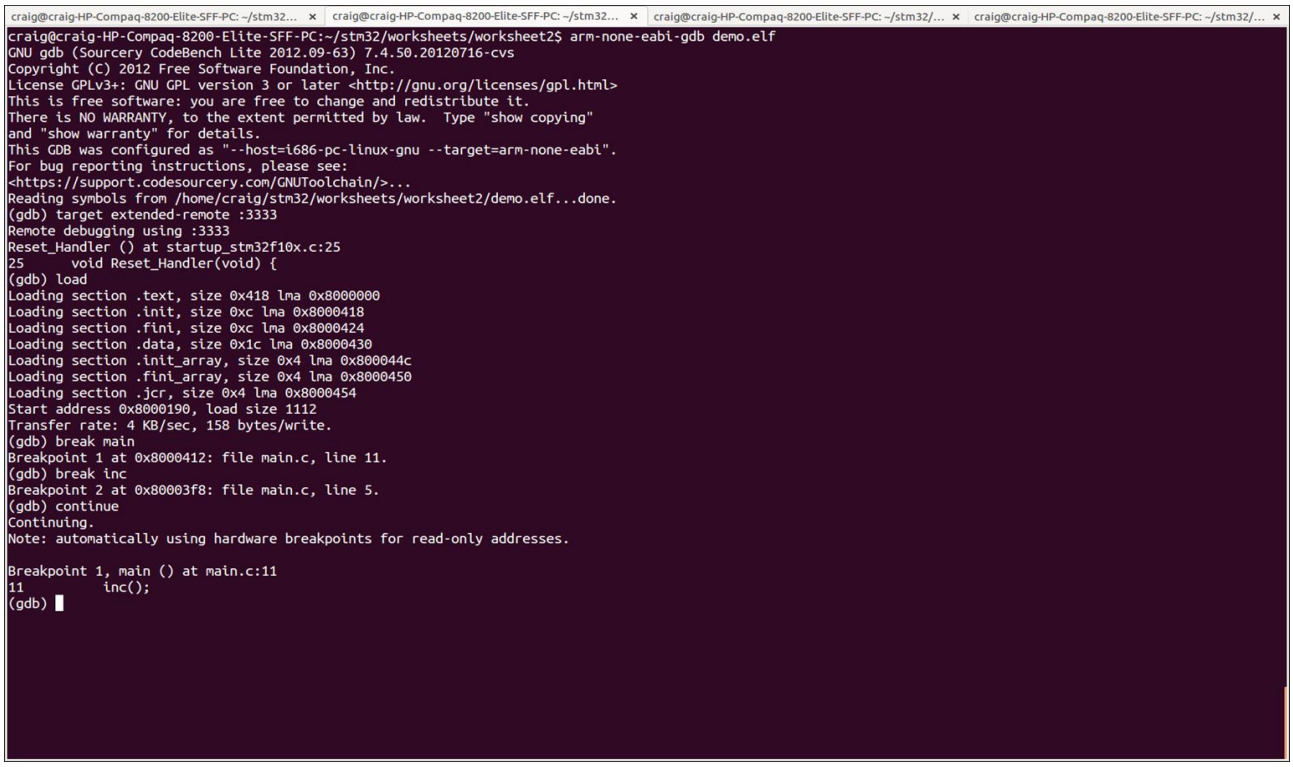
In the following section you might find on some systems gdb-multiarch isn't installed. This maybe because the system is using an earlier version of the cross compiler tools. In which case, you would need to use arm-none-eabi-gdb instead. If that doesn't exist it is possible that gdb-multiarch hasn't been installed and needs installing.

In another command prompt run the GNU debugger, **gdb-multiarch**, with the file **demo.elf** as its only argument. This assumes that you have run **make** in the worksheet2 directory first, otherwise you will get a file not found message. You should type the following commands into **gdb,**

```
%gdb-multiarch demo.elf
(gdb) target extended-remote :3333
(gbd) load
(gdb) break main
(gdb) break inc
(gdb) continue
gdb commands
```

gdb commands

The **(gdb)** is a prompt from **gdb**. The command **target** connects to the **openocd** deamon which is running on port 3333. We make sure that the various symbol tables are loaded. We then create 2 break points, telling the debugger where to stop the code, once in the function **main**, and again in the function **inc**. c**ontinue** tells the debugger to execute the program up to the break points.



```
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/worksheets/worksheet2$ arm-none-eabi-gdb demo.elf
GNU gdb (Sourcery CodeBench Lite 2012.09-63) 7.4.50.20120716-cvs
Copyright (C) 2012 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=i686-pc-linux-gnu --target=arm-none-eabi".
For bug reporting instructions, please see:
<https://support.codesourcery.com/GNUToolchain/>...
Reading symbols from /home/craig/stm32/worksheets/worksheet2/demo.elf...done.
(gdb) target extended-remote :3333
Remote debugging using :3333
Reset_Handler () at startup_stm32f10x.c:25
25      void Reset_Handler(void) {
(gdb) load
Loading section .text, size 0x418 lma 0x8000000
Loading section .init, size 0xc lma 0x8000418
Loading section .fini, size 0xc lma 0x8000424
Loading section .data, size 0x1c lma 0x8000430
Loading section .init_array, size 0x4 lma 0x800044c
Loading section .fini_array, size 0x4 lma 0x8000450
Loading section .jcr, size 0x4 lma 0x8000454
Start address 0x8000190, load size 1112
Transfer rate: 4 KB/sec, 158 bytes/write.
(gdb) break main
Breakpoint 1 at 0x8000412: file main.c, line 11.
(gdb) break inc
Breakpoint 2 at 0x80003f8: file main.c, line 5.
(gdb) continue
Continuing.
Note: automatically using hardware breakpoints for read-only addresses.

Breakpoint 1, main () at main.c:11
11          inc();
(gdb)
```

gdb output

Once we have loaded the code and connected the debugger we can run the program under the debugger's control and view various aspects of it. For example we can continually examine the value of the variable **i**.

```
(gdb) print i
(gdb) continue
....
```

Questions.

Use the **gdb** documentation to help with the following questions.

Change to main.c program to include another variable – **k** – that is also incremented. Rebuild, download and debug this program.

Set up a watchpoint on either variable **i** or **k.**

Print register values – for example **print /x $sp** which displays the stack pointer in hex.

Try using a breakpoint command that prints **i** and continues just before **main** calls **inc**.

Appendix A

Makefile

```
# name of executable - change this to change the executables name
ELF=demo.elf

# object files
OBJS=  $(STARTUP) main.o

# Tool path
TOOLROOT=/usr/bin

# Library path
LIBROOT=../STM32F10x_StdPeriph_Lib_V3.5.0

# Tools
CC=$(TOOLROOT)/arm-none-eabi-gcc
LD=$(TOOLROOT)/arm-none-eabi-gcc
AR=$(TOOLROOT)/arm-none-eabi-ar
AS=$(TOOLROOT)/arm-none-eabi-as

# Code Paths
DEVICE=$(LIBROOT)/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x
CORE=$(LIBROOT)/Libraries/CMSIS/CM3/CoreSupport
PERIPH=$(LIBROOT)/Libraries/STM32F10x_StdPeriph_Driver

# Search path for standard files
vpath %.c .

# Search path for perpheral library
vpath %.c $(CORE)
vpath %.c $(PERIPH)/src
vpath %.c $(DEVICE)

# Processor specific
PTYPE = STM32F10X_CL
LDSCRIPT = stm32f100.ld
STARTUP= startup_stm32f10x.o system_stm32f10x.o

# compilation flags for gdb
CFLAGS  = -O1 -g

# Compilation Flags
FULLASSERT = -DUSE_FULL_ASSERT
LDFLAGS+= -T$(LDSCRIPT) -mthumb -mcpu=cortex-m3
CFLAGS+= -mcpu=cortex-m3 -mthumb
CFLAGS+= -I. -I$(DEVICE) -I$(CORE) -I$(PERIPH)/inc -I.
CFLAGS+= -D$(PTYPE) -DUSE_STDPERIPH_DRIVER $(FULLASSERT)

# Build executable
$(ELF) : $(OBJS)
	$(LD) $(LDFLAGS) -o $@ $(OBJS)

# compile and generate dependency info
%.o: %.c
	$(CC) -c $(CFLAGS) $< -o $@

%.o: %.s
	$(CC) -c $(CFLAGS) $< -o $@
```

```
clean:
	rm -f $(OBJS)  $(ELF)

debug: $(ELF)
	gdb-multiarch $(ELF)

# pull in dependencies
-include $(OBJS)
```

Appendix B Linker description file

```
/*
  Linker script for STM32F10x_256K_64K
  modified from
  http://www.codesourcery.com/archives/arm-gnu/msg02972.html
  http://communities.mentor.com/community/cs/archives/arm-gnu/msg02972.html
*/

/*
  There will be a link error if there is not this amount of RAM free at the end.
*/

_Minimum_Stack_Size = 256;

ENTRY(Reset_Handler)

/* Memory Spaces Definitions */
MEMORY
{
  RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 64K
  FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 256K
}

__ram_start__ = ORIGIN(RAM);
__ram_size__  = LENGTH(RAM);
__ram_end__   = __ram_start__ + __ram_size__;
_estack = __ram_end__;

/* highest address of the user mode stack */
PROVIDE ( _Stack_Limit = _estack - _Minimum_Stack_Size );

/* Sections Definitions */
SECTIONS
{
  .text :
  {
     KEEP(*(.isr_vector))      /* Startup code */
     *(.text)                  /* code */
     *(.text.*)                /* remaining code */
     *(.rodata)                /* read-only data (constants) */
     *(.rodata.*)
     *(.glue_7)
     *(.glue_7t)
     *(.vfp11_veneer)
     *(.v4_bx)
     *(.ARM.extab* .gnu.linkonce.armextab.*)
  } >FLASH

  /* for exception handling/unwind - some Newlib functions (in
     common with C++ and STDC++) use this. */
  .ARM.extab :
  {
     *(.ARM.extab* .gnu.linkonce.armextab.*)
  } > FLASH

  __exidx_start = .;
  .ARM.exidx :
  {
     *(.ARM.exidx* .gnu.linkonce.armexidx.*)
  } > FLASH
```

```
__exidx_end = .;

. = ALIGN(4);
_etext = .;

/* This is used by the startup in order to initialize the .data secion */
_sidata = _etext;

/* This is the initialized data section
  The program executes knowing that the data is in the RAM
  but the loader puts the initial values in the FLASH (inidata).
  It is one task of the startup to copy the initial values from
  FLASH to RAM. */
.data  : AT ( _sidata )
{
   . = ALIGN(4);
   /* This is used by the startup in order to initialize the .data section */
   _sdata = . ;
   *(.data)
   *(.data.*)
   . = ALIGN(4);

   /* This is used by the startup in order to initialize the .data section */
   _edata = . ;
} >RAM

/* This is the uninitialized data section */
.bss :
{
   . = ALIGN(4);
   /* This is used by the startup in order to initialize the .bss section */
   _sbss = .;
   __bss_start__ = _sbss;
   *(.bss)
   *(.bss.*)
   *(COMMON)

   . = ALIGN(4);
   /* This is used by the startup in order to initialize the .bss section */
   _ebss = . ;
   __bss_end__ = _ebss;
} >RAM

PROVIDE ( end = _ebss );
PROVIDE ( _end = _ebss );
PROVIDE ( _exit = _ebss );
PROVIDE (_stackend = ORIGIN(RAM) + LENGTH(RAM) - _Minimum_Stack_Size);

/* This is the user stack section
  This is just to check that there is enough RAM left for the User mode stack
  It should generate an error if it's full.
*/
._usrstack :
{
   . = ALIGN(4);
   _susrstack = . ;
   . = . + _Minimum_Stack_Size ;
   . = ALIGN(4);
   _eusrstack = . ;
} >RAM

/* after that it's only debugging information. */
```

```
  /* remove the debugging information from the standard libraries */
/*
  DISCARD :
  {
    libc.a ( * )
    libm.a ( * )
    libgcc.a ( * )
  }
*/

  /* Stabs debugging sections.  */
  .stab         0 : { *(.stab) }
  .stabstr      0 : { *(.stabstr) }
  .stab.excl    0 : { *(.stab.excl) }
  .stab.exclstr  0 : { *(.stab.exclstr) }
  .stab.index    0 : { *(.stab.index) }
  .stab.indexstr 0 : { *(.stab.indexstr) }
  .comment      0 : { *(.comment) }

  /* DWARF debug sections.
     Symbols in the DWARF debugging sections are relative to the beginning
     of the section so we begin them at 0.  */

  /* DWARF 1 */
  .debug         0 : { *(.debug) }
  .line          0 : { *(.line) }

  /* GNU DWARF 1 extensions */
  .debug_srcinfo  0 : { *(.debug_srcinfo) }
  .debug_sfnames  0 : { *(.debug_sfnames) }

  /* DWARF 1.1 and DWARF 2 */
  .debug_aranges  0 : { *(.debug_aranges) }
  .debug_pubnames 0 : { *(.debug_pubnames) }

  /* DWARF 2 */
  .debug_info     0 : { *(.debug_info .gnu.linkonce.wi.*) }
  .debug_abbrev   0 : { *(.debug_abbrev) }
  .debug_line     0 : { *(.debug_line) }
  .debug_frame    0 : { *(.debug_frame) }
  .debug_str      0 : { *(.debug_str) }
  .debug_loc      0 : { *(.debug_loc) }
  .debug_macinfo  0 : { *(.debug_macinfo) }

  /* SGI/MIPS DWARF 2 extensions */
  .debug_weaknames 0 : { *(.debug_weaknames) }
  .debug_funcnames 0 : { *(.debug_funcnames) }
  .debug_typenames 0 : { *(.debug_typenames) }
  .debug_varnames  0 : { *(.debug_varnames) }
}
```

Appendix C A note on untarring files

Please think before untarring! You are going to need to work with the files so try to put them into sensible places not just in any random place you happen to be. As you work on future worksheets you will need some of the files to be in specific or known places. Having a hierarchy such as SES/worksheets/ would be good. Be careful of using the windowing file manager for creating directories as you might be tempted to create names like 'Secure Embedded', the spaces in this path name will play havoc with scripts in later worksheets.

In order to complete this section of the worksheet you will need to download the tar file with the example code in it: **worksheet2.tgz** which is either on my home page or on the course BB site. This file contains all of the code required to build the programs for this worksheet. Download the file and untar it, put it in a sub-directory in your mobile and embedded devices sub-directory. To untar first check that the archive isn't corrupt or empty with the **-t** (table of contents) command switch, the **zf** switches are for unzip (**z**) and the **f** flag means the next argument is a file name. You can check for further details on the command line arguments for **tar** with **man tar**.

%tar -tzf wotksheet2.tgz

You should see something like:

```
STM32F10x_StdPeriph_Lib_V3.5.0/
STM32F10x_StdPeriph_Lib_V3.5.0/Release_Notes.html
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/License.doc
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/Documentation/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/Documentation/CMSIS_Core.htm
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CMSIS debug support.htm
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport/core_cm3.h
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/CoreSupport/core_cm3.c
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/system_stm32f10x.c
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/Release_Notes.html
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/system_stm32f10x.h
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/stm32f10x.h
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_xl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_hd_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_ld.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_hd.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_ld_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_md.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_cl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/TrueSTUDIO/startup_stm32f10x_md_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_xl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_hd_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_ld.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_hd.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_ld_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_md.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_cl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/iar/startup_stm32f10x_md_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/startup_stm32f10x_xl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/startup_stm32f10x_hd_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/startup_stm32f10x_ld.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/startup_stm32f10x_hd.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/startup_stm32f10x_ld_vl.s
STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x/startup/gcc_ride7/startup_stm32f10x_md.s
--More--
```

tar output

Assuming the output looks correct then you can untar it with – **x** for extract:

%tar -xzf worksheet2.tgz

Ensure that you are in the correct directory when extracting. There are a lot of files and sub-directories, many will be redundant as they will be for other tool chains or systems/sub-systems we

aren't using. Once the archive has been untared and you are in the directory into which they have been extracted you should see 2 sub-directories - worksheet2 and STM32F10x_StdPeriph_Lib_V3.5.0, this is a directory with the ARM and ST Micro code, over the coming worksheets we will explore more and more of this in detail.